Background

Glossary

- **Harbor** — open-source framework we use to author tasks, run agents against them, and capture trajectories. Docs: harborframework.com/docs/getting-started.
- **Task** — a single problem with a verifiable outcome. Has an instruction, an environment, and a verifier that scores the agent's final state.
- **Environment** — the sandboxed world the agent acts in: container image, available tools, MCP servers, internet access, files on disk.
- **Agent** — the LLM-plus-scaffolding under test (e.g., Claude Code, Gemini CLI, Terminus-2). Takes the instruction, acts in the environment, produces a final state.
- **Verifier** — code that inspects the post-execution state and emits a reward (in Harbor: a `tests/test.sh` driver that writes to `/logs/verifier/reward.{txt,json}`).
- **Reward** — the score, scalar or multi-criterion. The training signal.
- **Trajectory** — the captured execution: messages, tool calls, observations, reasoning, final output. Harbor emits these as ATIF JSON automatically when you `harbor run`.
- **Headroom** — the gap between current SOTA performance on a capability and what's achievable. The thing RL data is supposed to close.
- **pass@k** — fraction of tasks where at least one of `k` model attempts passes the verifier. We use **pass@3** as the difficulty bar in this exercise.

The Ask

Design a mini eval for **data science tasks**. You decide how narrow or broad the scope should be — a tightly focused eval is often more informative than a sprawling one across all of data science. How you slice *is* part of what we're evaluating.

Two hard requirements:

- **Real distribution.** Tasks must be representative of the actual work data scientists do in industry. public Kaggle notebooks, real ML / DS workflows, production-style ETL pipelines, recent DS benchmark papers, etc.
- **Sufficient headroom.** The final set should target The final set should target < 30% pass@3 against gemini-3.5-flash.

The deliverable is 5–10 Harbor-format tasks that meet both bars, plus a writeup defending your choices.

Deliverables

A single zip with this layout:

```
samples/                    # 5–10 Harbor-format data-science tasks
logs/                       # `harbor run` output for each task (≥ 3 trials)
report/                     # Single report, your choice of format
```

`samples/` — 5–10 fully built Harbor tasks. You're encouraged to pilot more candidate tasks and curate down — that selection process is itself a useful signal.

`logs/` — for each task, capture **at least 3 trials** of `harbor run -p <task-path> -a terminus-2 -m gemini/gemini-3.5-flash` so you (and we) can compute pass@1 and pass@3.

`report/` — single report, your choice of format (md, pdf, html, slides — whatever reads cleanly). It should answer the following questions:

1. **Distribution.** How did you choose what to measure, and why this slice? Where does your task distribution come from? What's deliberately in scope, and what's not?
2. **Difficulty profile.** What's the per-task and aggregate pass@1 / pass@3 against `gemini-3.5-flash`? What does the difficulty curve look like, and what kinds of failures dominate? **Plots are strongly encouraged**.
3. **Research awareness.** What benchmarks, papers, blog posts, or production tools did you look at while designing this? What are the takeaways?
4. **Scale plan.** If we wanted to grow this from 10 to 1,000 tasks for training data, how would you do it? Public data sources you'd lean on, augmentation strategy, QA loop etc.
5. **Bonus — failure analysis.** For a handful of tasks the model fails on, diagnose *why*. Pull from the trajectories: where did the model go off-rails — misreading the data, wrong tool choice, arithmetic slip? Use this to argue failures are coming from genuine task difficulty, not from task-design bugs (ambiguous instructions, broken environment, over-strict verifier)

Submission

Zip up `samples/`, `logs/`, and `report/` and email to `careers@abundant.ai` with subject `Research Take-Home — <Your Name>`.

Next step: we'll review your submission and get back to you with our feedback.

Other resources

Hints for finding headroom

You're not graded on *which* domain you pick — you're graded on whether you find a real failure mode and design samples that isolate it. Three angles that consistently surface signal:

- **Mine the literature.** Read recent papers, blog posts, and lab releases in the space (last ~6 months). Try recreating tasks from their evals — frontier benchmarks are a cheat-code for finding failure

modes that haven't been solved yet, and SOTA plateaus tell you where to dig. As quick starting points: CVE-Bench and the Harbor dataset hub.

- **Push on messy, multi-modal I/O.** Real knowledge work isn't clean text. Try inputs that mix forensic artifacts, PDFs, Git patches, etc. Try outputs that have to fill structured schemas, edit existing artifacts in place, or interleave reasoning with citations.
- **Experiment with different types of verifiers**. Deterministic vs LLM-as-a-Judge, binary vs fractional reward. Justify your decision.

Technical Instructions

Step by step

The fastest way to get oriented with Harbor is to run the example task we've included alongside this brief in three configurations, then mirror the same flow on your own task.

1. **Unzip the example task** and skim the four key files: `instruction.md`, `task.toml`, `environment/Dockerfile`, `tests/test.sh` (plus the optional `solution/solve.sh`).
2. **Run it with the Oracle agent.** Oracle executes `solution/solve.sh` against the environment — a sanity-check that the task is actually solvable.

```
harbor run -p <example-task> -a oracle
```

Expected reward: `1`. If not, the solution or verifier is broken.

3. **Run it with the Nop agent.** Nop does nothing — it just spins up the environment and exits. This proves the verifier doesn't pass on inaction.

```
harbor run -p <example-task> -a nop
```

Expected reward: `0`. If you see `1`, your verifier is already satisfied by the default state and the task isn't measuring anything.

4. **Run it with the model under test.** This is the configuration you'll capture into `logs/` for your submission.

```
harbor run -p <example-task> -a terminus-2 -m gemini/gemini-3.5-flash
```

5. **IMPORTANT: Inspect agent trajectory.** Inspect `jobs/<job>/<trial>/agent/trajectory.json` and `verifier/reward.txt` to see the real model behavior.

Quality-check each task with `harbor check`

Before submitting, run each of your tasks through harbor check to catch common authoring problems (under-specified instructions, gameable verifiers, anti-cheat holes, etc.). Use the TB3 task-implementation rubric as an example — it codifies what "good task" means in practice:

```
harbor check <task-path> -r
https://github.com/harbor-framework/terminal-bench-3/blob/main/rubrics/task-implementation.toml
```

Trajectories are produced automatically. When you `harbor run -p <task-path> -a terminus-2 -m gemini/gemini-3.5-flash`, Harbor emits a `jobs/<job>/<trial>/` directory containing the ATIF-formatted `agent/trajectory.json`, the verifier output under `verifier/`, and a top-level `result.json`. You don't write trajectories by hand — you submit the captured output.

Trajectory inspection. Check the agent trajectory to verify that the failure is from genuine task difficult (e.g. reasoning error, tool call failure, misunderstanding instruction etc) and not task design issues (e.g. cheating, missing instruction, verifier design issue etc)

Other practicals

- **Gemini API.** `gemini-3.5-flash` is the model under test. Key: `GEMINI_API_KEY=AIzaSyDCUXJyIbOhK_X-PEpK7e5GkHYysRv2obg`.
- **Coding agents.** Use Claude Code or Codex aggressively — this role doesn't reward hand-coding what an agent could write faster. Tell us in the report which loops you automated.
- **Out of scope.** UI, deployment, scaling the verifier infra. Don't pad the report. Don't ship un-edited AI boilerplate.